1 Create a Sequential model for classifying MNIST digits with the following architecture: Flatten input layer for shape (28,28), Dense layer with 128 neurons and ReLU activation, Dropout layer with rate 0.3, Dense output layer with 10 neurons and softmax activation. Compile the model using Adam optimizer and categorical crossentropy loss.

```
from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Dropout, Flatten

from tensorflow.keras.optimizers import Adam

model = Sequential([

   Flatten(input_shape=(28, 28)),

   Dense(128, activation='relu'),

   Dropout(0.3),

   Dense(10, activation='softmax')

])

model.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['accuracy'])

model.summary()
```

2. Create a CNN model for grayscale images of shape (28,28,1) with: Conv2D layer with 32 filters, 3x3 kernel, ReLU; MaxPooling2D with pool size 2x2; Conv2D layer with 64 filters, 3x3 kernel, ReLU; Flatten and Dense layer with 128 neurons, ReLU; Output Dense layer with 10 neurons, softmax.

```
from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

cnn = Sequential([

   Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),

   MaxPooling2D((2,2)),

   Conv2D(64, (3,3), activation='relu'),

   Flatten(),

   Dense(128, activation='relu'),

   Dense(10, activation='softmax')

]) cnn.summary()
```

3 Given a pre-trained Sequential model, add a new Dense layer of 64 neurons before the output and compile the model.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
pretrained = Sequential([
    Dense(128, activation='relu', input_shape=(64,)),
    Dense(10, activation='softmax')
])
# Add new Dense layer before output
pretrained.pop()  # remove output
pretrained.add(Dense(64, activation='relu'))
pretrained.add(Dense(10, activation='softmax'))
pretrained.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

4 Explain in code comments the difference between using sigmoid vs softmax for the output layer. Then implement an example with 3-class classification.

```
# Sigmoid → used for binary output (0/1)
# Softmax → used for multi-class classification (probabilities sum to 1)
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
model = Sequential([
    Dense(64, activation='relu', input_shape=(20,)),
    Dense(3, activation='softmax')  # for 3-class classification
])
```

5 Create a functional API model that takes two inputs: Input1 of shape (32,) and Input2 of shape (32,). Concatenate them, pass through a Dense layer of 64 neurons, then output a single neuron with sigmoid activation.

```
from tensorflow.keras.layers import Input, Dense, Concatenate
from tensorflow.keras.models import Model
input1 = Input(shape=(32,))
input2 = Input(shape=(32,))
merged = Concatenate()([input1, input2])
dense = Dense(64, activation='relu')(merged)
output = Dense(1, activation='sigmoid')(dense)
model = Model(inputs=[input1, input2], outputs=output)
model.summary()
```

6 Load the CIFAR-10 dataset, normalize images to range 0–1, and one-hot encode the labels.

```
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_train, x_test = x_train/255.0, x_test/255.0
y_train, y_test = to_categorical(y_train), to_categorical(y_test)
```

7 Given a dataset X of shape (1000, 28, 28), reshape it appropriately for a CNN input

```
import numpy as np
X = np.random.rand(1000, 28, 28)
X_cnn = X.reshape(1000, 28, 28, 1)
```

8 Write code to split a dataset into 70% training, 15% validation, and 15% testing using sklearn.

```
from sklearn.model_selection import train_test_split
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5)
```

9 Apply data augmentation on image data using ImageDataGenerator with horizontal flips, rotations of 15 degrees, and width/height shifts of 0.1.

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)
```

10 Visualize 5 random images from your dataset with their corresponding labels using matplotlib.

```
import matplotlib.pyplot as plt
import numpy as np
indices = np.random.choice(len(x_train), 5)
for i, idx in enumerate(indices):
    plt.subplot(1,5,i+1)
    plt.imshow(x_train[idx])
    plt.title(np.argmax(y_train[idx]))
    plt.axis('off')
plt.show()
```

11. Train a Sequential model on training data for 10 epochs, using validation data for evaluation. Include EarlyStopping if validation loss doesn’t improve for 3 epochs.

```
from tensorflow.keras.callbacks import EarlyStopping

es = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)

model.fit(x_train, y_train, validation_data=(x_val, y_val), epochs=10, callbacks=[es])
```

12. Implement ModelCheckpoint to save the best model during training based on validation accuracy.

```
from tensorflow.keras.callbacks import ModelCheckpoint

checkpoint = ModelCheckpoint('best_model.h5', save_best_only=True,
monitor='val_accuracy', mode='max')

model.fit(x_train, y_train, validation_data=(x_val, y_val), epochs=10, callbacks=[checkpoint])
```

13. Plot the training and validation loss curves for a trained model and explain how to identify overfitting from the graph.

```
import matplotlib.pyplot as plt

plt.plot(history.history['loss'], label='Train Loss')

plt.plot(history.history['val_loss'], label='Val Loss')

plt.legend()

plt.show()

# Overfitting → when val_loss increases while train_loss keeps decreasing
```

14. Evaluate a trained model on test data and print both loss and accuracy.

```
loss, acc = model.evaluate(x_test, y_test)

print("Test Loss:", loss)

print("Test Accuracy:", acc)
```

15. Given training and validation loss arrays: train_loss = [0.8, 0.5, 0.3, 0.2] and val_loss = [0.9, 0.6, 0.4, 0.5], write code to detect overfitting and explain why it occurs.

```
train_loss = [0.8, 0.5, 0.3, 0.2]
val_loss = [0.9, 0.6, 0.4, 0.5]
if val_loss[-1] > val_loss[-2]:
    print("Model is overfitting.")
# Overfitting occurs when validation loss rises even though training loss falls.
```

16. Given a pre-trained CNN, write code to freeze all layers except the last Dense layer, then compile it for fine-tuning.

```
for layer in cnn.layers[:-1]:
    layer.trainable = False
cnn.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

17. Extract the output of an intermediate layer (second Dense layer) for a single input sample using Keras functional API.

```
from tensorflow.keras.models import Model
import numpy as np
intermediate_model = Model(inputs=model.input, outputs=model.layers[2].output)
sample = np.random.rand(1, 28, 28, 1)
output = intermediate_model.predict(sample)
```

18. Implement a custom loss function for mean squared error and use it in compiling a model.

```
import tensorflow as tf
def custom_mse(y_true, y_pred):
    return tf.reduce_mean(tf.square(y_true - y_pred))
model.compile(optimizer='adam', loss=custom_mse)
```

19. Save a trained model to disk in both HDF5 and SavedModel formats. Then write code to load it back.

```
# Save
model.save('model.h5')          # HDF5 format
model.save('saved_model/')      # SavedModel format
# Load
from tensorflow.keras.models import load_model
m1 = load_model('model.h5')
m2 = load_model('saved_model/')
```

20. Apply softmax manually on the tensor logits = [2.0, 1.0, 0.1] using numpy and compare it with TensorFlow's softmax output.

```
import numpy as np
import tensorflow as tf
logits = np.array([2.0, 1.0, 0.1])
softmax_np = np.exp(logits) / np.sum(np.exp(logits))
softmax_tf = tf.nn.softmax(logits)
print("NumPy Softmax:", softmax_np)
print("TensorFlow Softmax:", softmax_tf.numpy())
```